

Deterministic Metadata Obfuscation against Deep Learning QUIC Traffic Classification: A Privacy–Cost Pareto Study on Backbone Traces

Abhijat Bhat, Aditya Narayan, Akash Sivaprasath

Department of Computer Science and Engineering, RV College of Engineering
Bengaluru, India

Abstract—Encrypted transport protocols such as QUIC hide application payloads, yet flow-level metadata—packet sizes, directions, and inter-packet times—remain observable to network operators and passive adversaries. Deep learning classifiers can exploit this metadata to infer which application produced a flow. We study *deterministic* metadata obfuscation as a countermeasure on real CESNET-QUIC22 backbone traces. Using a temporally held-out test week (49,305 flows, 64 application classes), we train and freeze two attack architectures—a masked Transformer encoder and a CNN-BiLSTM—and evaluate seven obfuscation policies spanning timing jitter, linear size padding, and MTU padding, with manifest-derived bandwidth and latency overhead. On a masked Transformer attacker, clean traffic yields 74.4% macro F1 (77.8% accuracy). Moderate Laplace jitter with only 11 ms mean latency overhead (*jitter_low*) reduces macro F1 by 1.7 percentage points while preserving 76.8% accuracy; the degradation is statistically significant (McNemar $p \approx 5.8 \times 10^{-22}$) but small in magnitude. Strong jitter and MTU padding achieve near-random performance at very different cost points. Transformers remain more robust than CNN-BiLSTMs under jitter, while both architectures collapse under MTU obfuscation. We release a reproducible pipeline and formal Pareto analysis to guide proxy deployment.

Index Terms—QUIC, traffic classification, metadata obfuscation, privacy, Pareto frontier, deep learning

I. INTRODUCTION

Application-layer encryption and modern transport protocols have reduced the effectiveness of deep packet inspection (DPI) for identifying user activities. QUIC, now widely deployed for web and video services, encrypts most protocol headers and all payloads. Nevertheless, *flow metadata*—sequence of packet sizes, directions, and inter-packet times (IPTs)—is still visible on backbone and access links. Prior work shows that machine learning models can infer application classes from such metadata with high accuracy, creating privacy risks for users behind nation-state, ISP, or enterprise monitoring.

A natural defense is to route traffic through an anonymizing proxy or middlebox that *re-shapes* metadata before it leaves the user’s trust domain. Unlike encryption, obfuscation does not hide that a flow exists; instead it degrades the statistical fingerprints exploited by classifiers. The defender faces a multi-objective problem: maximize privacy (reduce classifier confidence) while minimizing bandwidth and latency overhead.

This paper presents an end-to-end measurement study on the CESNET-QUIC22 backbone dataset [1]. We make four contributions:

- 1) A reproducible pipeline from raw QUIC flows to frozen deep learning attackers and deterministic obfuscators, with temporal train/test separation (weeks W-2022-44 / W-2022-45).
- 2) A systematic evaluation of seven obfuscation policies with manifest-derived bandwidth and latency costs, plus formal Pareto dominance analysis.
- 3) Evidence that *jitter_low* is a practical operating point: ≈ 1.7 percentage point macro-F1 loss at 11 ms mean latency overhead and zero bandwidth expansion—validated with bootstrap confidence intervals and paired significance tests.
- 4) A comparison of CNN-BiLSTM vs. masked Transformer robustness under the same defenses, showing Transformers resist timing jitter better while both fail under aggressive MTU padding.

Section II reviews related work. Section III states the threat model. Section IV describes data, models, and obfuscators. Section V presents results; Section VI discusses deployment implications; Section VII concludes.

II. RELATED WORK

Encrypted traffic classification. Early work inferred application behavior from flow statistics and hand-crafted features [2], [3]. Deep learning replaced manual feature design with end-to-end models over packet sequences—1D CNNs, recurrent networks, and attention-based encoders [4]–[8]. QUIC-specific backbone datasets such as CESNET-QUIC22 enable realistic evaluation rather than synthetic traces [1], [9].

Traffic morphing and obfuscation. Defenses include padding to fixed sizes, adaptive packet scheduling, dummy traffic, and record-level padding in TLS/QUIC stacks [10], [11]. Website fingerprinting literature formalized attack–defense tradeoffs for morphing proxies [12], [13]. Our work focuses on *deterministic*, proxy-realizable transforms—linear block padding, MTU padding, and Laplace timing jitter—whose overhead can be measured directly from transformed traces.

Privacy–utility tradeoffs. Prior studies often report accuracy drops without joint analysis of bandwidth/latency

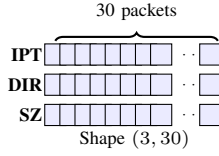


Fig. 1: Fixed-length flow representation fed to attack models. Channel 0 (IPT) is z-scored; channel 1 is direction; channel 2 is packet size normalized by MTU.

cost [14]. We adopt multi-objective framing and compute Pareto-efficient operating points, reporting bootstrap confidence intervals and paired significance tests [15], [16].

Gap. Few works jointly (i) evaluate modern QUIC backbone data, (ii) compare Transformer vs. CNN sequence models under identical obfuscation, and (iii) report formal Pareto frontiers with statistical confidence on held-out temporal test data. We address this gap with a fully scripted open pipeline.

III. THREAT MODEL AND ASSUMPTIONS

Adversary. We consider a passive adversary (e.g., ISP or backbone observer) who records the first 30 packets of each QUIC flow and extracts three metadata channels per packet: normalized IPT, direction (client-to-server vs. server-to-client), and packet size. The adversary does not decrypt payloads or QUIC headers protected by TLS [17]. The adversary knows the feature extraction pipeline and has trained a supervised multi-class classifier on historical labeled data from the same network environment.

Defender. The defender operates an inline proxy that applies deterministic obfuscation to outgoing (and optionally incoming) packets before they traverse the monitored link. The defender cannot hide flow existence or volume at the coarsest level; the goal is to reduce *application-level* inferability.

Out of scope. We do not model adaptive adversaries who retrain on obfuscated traces (see Section VI), active attacks, or colluding endpoints. We evaluate a single held-out test week; cross-network generalization is future work.

Evaluation ethics. Experiments use published anonymized backbone traces under CESNET-QUIC22 terms of use; no live user traffic is collected in this study.

IV. METHODOLOGY

A. Dataset and preprocessing

We use CESNET-QUIC22-XS [1], a public QUIC traffic corpus collected on a national research and education backbone. Flows are labeled by application class. We train on calendar week W-2022-44 and test on the subsequent week W-2022-45 (strict temporal holdout). Rare classes with fewer than 200 training samples are removed, yielding 64 classes. Each flow is represented as a fixed-length tensor of shape (3, 30) (Fig. 1): channel 0 is z-scored IPT (index 0 never jittered, following CESNET convention), channel 1 is direction, channel 2 is packet size normalized by MTU (1500B). The test set contains 49,305 flows.

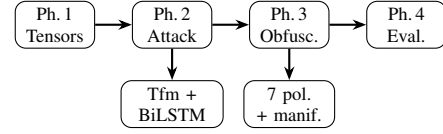


Fig. 2: Reproducible four-phase pipeline. Attack weights are frozen before defense evaluation.

B. End-to-end pipeline

Fig. 2 summarizes the experimental workflow. Phase 1 exports normalized tensors; Phase 2 trains attack models; Phase 3 applies obfuscators and logs per-flow overhead; Phase 4 evaluates frozen checkpoints and computes Pareto and statistical summaries.

C. Attack models

We train two architectures to convergence on identical splits:

- **CNN-BiLSTM:** 1D convolutions over the packet sequence followed by bidirectional LSTM pooling; selected hyperparameters from a six-run sweep (learning rate 10^{-3} , batch size 1024, weight decay 0.01).
- **Masked Transformer:** multi-head self-attention encoder with padding mask over variable valid lengths; $d_{\text{model}} = 160$, trained with masked loss (production checkpoint used for primary results).

Checkpoints are frozen before any defense evaluation. Primary metrics are *macro-averaged F1* (class-imbalance robust) and accuracy. Chance performance is 1.56% (uniform over 64 classes).

D. Obfuscation policies

Obfuscators apply deterministic transforms with logged per-flow overhead:

- **Jitter** (low / medium / high): add Laplace-distributed delay to packets with index > 0 ; zero bandwidth cost, mean latency overhead 11 ms, 55 ms, and 221 ms respectively on the test manifest.
- **Linear-128:** round each packet size up to the next 128 B block ($\approx 17.4\%$ mean bandwidth overhead).
- **Linear-128 + jitter_medium:** compose padding and moderate jitter.
- **MTU:** pad every packet to 1500 B ($\approx 274\%$ bandwidth overhead); MTU + jitter_medium combines both mechanisms.

Seven obfuscated test tensors plus a clean baseline are evaluated by reloading weights without retraining—modeling a defender who deploys obfuscation against a fixed, already-trained attacker. Fig. 3 illustrates the three mechanism families.

E. Metrics and statistics

Privacy is reported as classifier macro F1 and accuracy on obfuscated test data; **cost** as mean bandwidth overhead (%) and mean added latency (ms) from the obfuscation manifest.

Pareto analysis: a setting A dominates B if A is no worse on both cost and privacy loss and strictly better on at least one

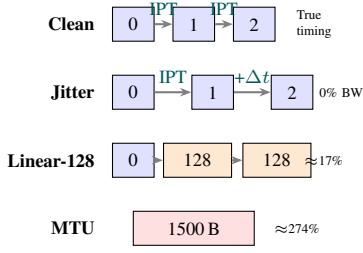


Fig. 3: Deterministic obfuscation on flow metadata (schematic). Jitter shifts inter-packet times for indices > 0 ; linear-128 rounds sizes upward; MTU pads every packet to 1500 B. Index 0 IPT is never jittered (CESNET convention).

TABLE I: Transformer test results: macro F1 (accuracy), privacy loss, and overhead.

Setting	F1 (Acc)	$\Delta F1$	BW%	Lat(ms)
baseline	74.4 (77.8)	0.0	0.0	0.0
jitter_high	47.7 (54.7)	26.8	0.0	220.8
jitter_low	72.7 (76.8)	1.7	0.0	11.0
jitter_med	63.0 (68.8)	11.4	0.0	55.2
linear128	62.0 (59.1)	12.4	17.4	0.0
lin128+j_med	50.0 (52.3)	24.4	17.4	55.2
mtu	0.3 (2.1)	74.1	274.0	0.0
mtu+j_med	0.4 (3.0)	74.0	274.0	55.2

axis. We report frontiers for bandwidth–accuracy and latency–accuracy views.

Tier C statistics: non-parametric bootstrap ($n = 2000$) yields 95% CIs for F1 and accuracy [15]. Paired bootstrap and McNemar’s test [16] compare baseline vs. *jitter_low* on matched flows. Channel ablation masks input channels at inference without retraining.

All experiments use the open-source scripts in Phases 1–4 of the accompanying repository; Appendix A lists checkpoints and commands.

V. RESULTS

All numbers below refer to the masked Transformer on the held-out test week unless Table III states otherwise.

A. Overall privacy–cost tradeoff

Table I summarizes macro F1 (with accuracy in parentheses), privacy loss (Δ macro F1 vs. baseline), and manifest overhead. Clean baseline achieves 74.4% macro F1. Timing jitter monotonically reduces inferability with increasing latency cost and *zero* bandwidth expansion. *jitter_low* loses only 1.7 percentage points of macro F1 while *jitter_high* loses 26.8 points at 221 ms latency. Linear-128 padding alone reduces macro F1 by 12.4 points at 17.4% bandwidth cost; MTU padding collapses classification to near chance (0.3% macro F1) at 274% bandwidth overhead.

Fig. 4 visualizes macro F1 across settings with a chance baseline at 1.56%. The recommended deployable point *jitter_low* is highlighted in the plotting pipeline.

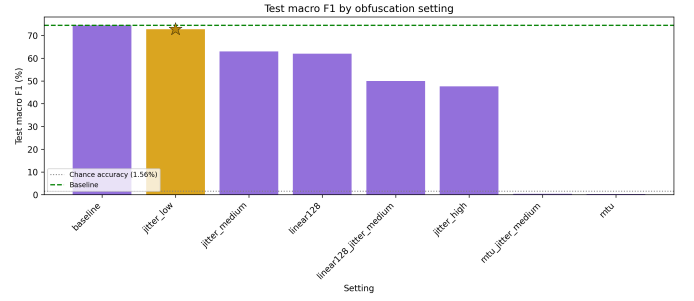


Fig. 4: Macro F1 by defense setting (Transformer, test week). Horizontal line: chance level (1/64). Star marks *jitter_low*.

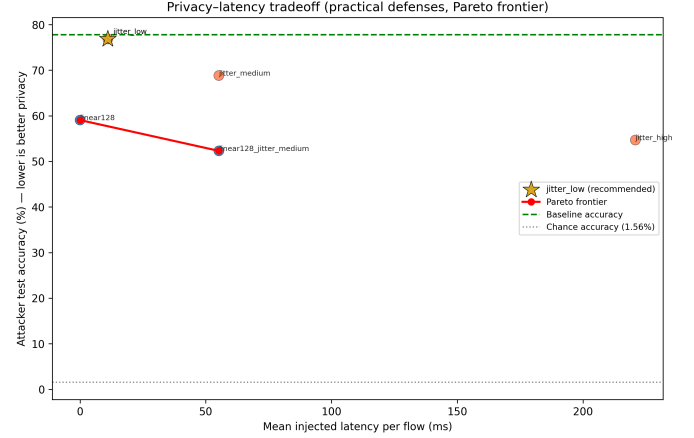


Fig. 5: Latency overhead vs. accuracy (practical view, MTU excluded).

B. Pareto frontiers

On the **latency–accuracy** plane (Fig. 5), jitter tiers form a clear frontier from *jitter_low* to *jitter_high*: higher latency buys substantially lower accuracy. Combined padding+jitter points lie off the jitter-only frontier when latency is held low.

On the **bandwidth–accuracy** plane (Fig. 6), all jitter settings share 0% bandwidth overhead; formally, only *jitter_high* and *linear128_jitter_medium* are non-dominated among non-MTU policies. Because jitter tiers are tied on bandwidth, we treat latency as the operative cost axis for timing defenses (Section VI).

Fig. 7 ranks settings by a dual privacy–cost score used in our plotting scripts; *jitter_low* ranks among the best latency-bounded options.

C. Statistical significance of jitter_low

Table II lists bootstrap 95% confidence intervals (2000 resamples, $n = 49,305$ test flows). For *jitter_low*, macro F1 lies in $[72.0\%, 73.4\%]$ and accuracy in $[76.5\%, 77.2\%]$. Paired comparison to baseline gives mean accuracy drop 0.93 percentage points (95% CI $[0.74, 1.12]$). McNemar’s test on discordant pairs ($b=1373$, $c=913$) yields $p \approx 5.8 \times 10^{-22}$ —the drop is statistically significant but modest compared with stronger defenses.

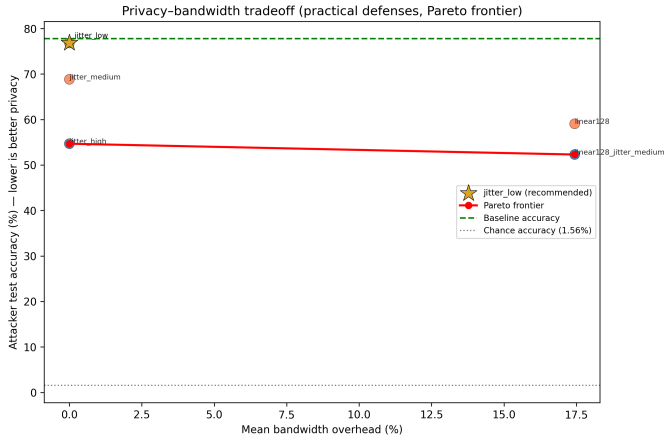


Fig. 6: Bandwidth overhead vs. accuracy (practical view). Jitter tiers overlap at 0% bandwidth.

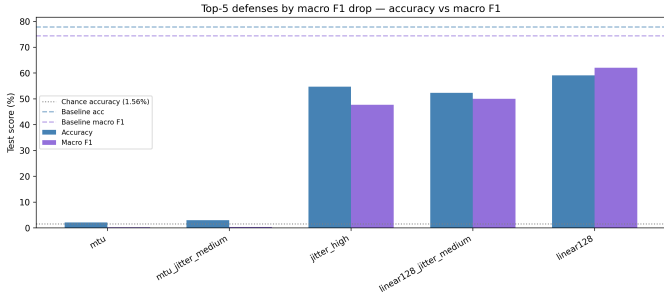


Fig. 7: Top-five settings under combined privacy-cost ranking.

D. Architecture comparison

Table III and Figs. 8–9 summarize CNN-BiLSTM vs. Transformer behavior. On clean traffic the Transformer leads by ≈ 5.0 percentage points accuracy (77.8% vs. 72.8%). Under *jitter_low* the gap widens slightly (≈ 5.9 points). Under *jitter_high* the Transformer retains 54.7% accuracy vs. 36.0% for BiLSTM—sequential models are more fragile to timing noise. Interestingly, BiLSTM exceeds Transformer accuracy on *linear128* alone (66.8% vs. 59.1%), suggesting convolutional local filtering partially compensates for size quantization artifacts. Both architectures fall to $\approx 2\text{--}3\%$ accuracy under MTU padding.

E. Channel ablation

Table IV reports inference-time channel masking. Single-channel models (IPT-only or size-only) fall to near chance; direction alone reaches 3.1% accuracy. The full 3-channel tensor is required for strong classification. IPT and direction channels are unchanged by jitter by design, yet jitter still degrades the full model, confirming cross-channel temporal correlation exploited by the Transformer.

F. Per-class vulnerability

Worst-class analysis shows persistent failure modes on low-support applications (e.g., classes 52, 58, 48) with $F1 < 0.4$

TABLE II: Bootstrap 95% CIs for selected settings (Transformer).

Setting	Acc	Acc CI	F1	F1 CI
baseline	77.8%	77.4–78.1	74.4%	73.7–75.0
jitter_low	76.8%	76.5–77.2	72.7%	72.0–73.4
jitter_med	68.8%	68.4–69.2	63.0%	62.2–63.7
jitter_high	54.7%	54.3–55.1	47.7%	46.9–48.3
mtu	2.1%	2.0–2.3	0.3%	0.27–0.32

TABLE III: Architecture comparison on test data (accuracy / macro F1, %).

Setting	Tfm Acc	LSTM Acc	Tfm F1	LSTM F1
baseline	77.8	72.8	74.4	67.4
jitter_high	54.7	36.0	47.7	29.6
jitter_low	76.8	71.0	72.7	65.1
jitter_med	68.8	57.9	63.0	50.4
linear128	59.1	66.8	62.0	63.1
lin128+j_med	52.3	50.6	50.0	43.9
mtu	2.1	2.9	0.3	1.6
mtu+j_med	3.0	2.6	0.4	1.2

even on clean data; *jitter_low* slightly degrades but does not fix these structural errors. MTU obfuscation zeroes recall for most classes—consistent with near-random aggregate metrics.

VI. DISCUSSION

Choosing an operating point. No single metric captures deployability. *jitter_high* formally lies on the bandwidth-accuracy Pareto frontier but imposes 221 ms mean latency—unacceptable for interactive QUIC. MTU padding maximizes privacy but roughly quadruples bytes sent—infeasible for backbone operators. *jitter_low* occupies a sweet spot: statistically distinguishable from baseline yet only 1.7 macro-F1 points lower, with 11 ms latency and no bandwidth penalty. We recommend it as the default proxy policy when bandwidth is constrained; stronger jitter tiers suit delay-tolerant bulk flows.

Transformer vs. BiLSTM. Global attention explains higher robustness to timing perturbation: distant packets remain correlated even when local IPT order is noised. BiLSTM’s sequential inductive bias amplifies jitter effects (*jitter_high* gap ≈ 18.7 percentage points accuracy). For padding-only defenses the ranking reverses—BiLSTM slightly wins on *linear128*—so defender-attacker co-design matters. Reporting a single “best architecture” without specifying defense is misleading.

Limitations. (i) Attackers are not retrained on obfuscated data—an adaptive adversary would partially recover accuracy. (ii) Single-network, single-month evaluation; seasonal drift and cross-ASN generalization are unverified. (iii) Deterministic obfuscation is predictable; adversaries who model the proxy transform may craft robust features. (iv) Class labels in per-class reports are numeric IDs; mapping to application names requires the released label encoder.

Future work. Adaptive attacker retraining, composable stochastic defenses, integration with QUIC PADDING frames and DATagram coalescing, and live testbed validation on a QUIC proxy (MASQUE/Oblivious HTTP) are natural extensions.

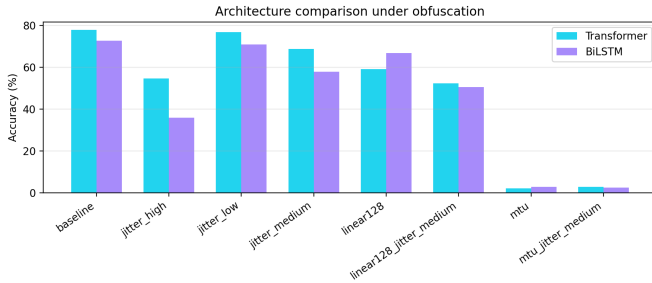


Fig. 8: Transformer vs. CNN-BiLSTM accuracy across defense settings.

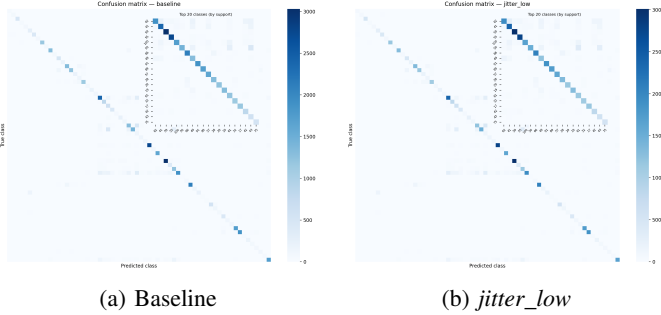


Fig. 9: Confusion matrices (top-20 classes inset). *jitter_low* preserves structure with modest spread.

VII. CONCLUSION

We presented a reproducible study of deterministic QUIC metadata obfuscation against deep learning traffic classifiers on CESNET-QUIC22 backbone data. Seven obfuscation policies expose a structured privacy–cost landscape: timing jitter buys privacy at latency cost with zero bandwidth expansion; padding buys stronger privacy at heavy bandwidth cost. A masked Transformer attacker achieves 74.4% macro F1 on clean held-out flows; *jitter_low* preserves 72.7% macro F1 at 11 ms overhead—a practical proxy configuration supported by bootstrap and McNemar tests. Transformers outperform CNN-BiLSTMs under jitter, while MTU padding defeats both. Our Pareto analysis and open pipeline aim to support evidence-based deployment of metadata defenses in QUIC-heavy networks.

REFERENCES

- [1] T. Benčič, M. Habermál, T. Čejka, and P. Ryšavý, “CESNET-QUIC22: A large one-month QUIC network traffic dataset from backbone lines,” in *Proceedings of the 19th International Conference on Network and Service Management (CNSM)*, 2023.
- [2] C. V. Wright, L. Ballard, S. E. Coull, F. Monrose, and G. M. Masson, “Spot me if you can: Uncovering shared profiles in large-scale network data sets,” in *USENIX Security Symposium*, 2009.
- [3] B. Anderson and D. McGrew, “Identifying encrypted malware traffic with contextual flow data,” in *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, 2016.
- [4] W. Wang, M. Zhu, X. Zeng *et al.*, “End-to-end encrypted traffic classification with one-dimensional convolution neural networks,” in *IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2017.

TABLE IV: Channel ablation (Transformer accuracy, test set).

Active channels	Baseline	<i>jitter_low</i>
All (IPT, DIR, SIZE)	77.8%	76.8%
IPT only	0.17%	0.17%
DIR only	3.1%	3.1%
SIZE only	0.17%	0.17%
IPT + DIR	3.7%	3.7%

- [5] S. Rezaei, X. Liu, and J. Liu, “Deep learning for encrypted traffic classification: An overview,” *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.
- [6] M. Lotfollahi, M. Zade, K. Osatuyi *et al.*, “Deep learning for encrypted traffic classification: An overview,” *IEEE Communications Magazine*, vol. 58, no. 3, pp. 76–83, 2020.
- [7] Z. Lin, Z. Wang, Y. Liu *et al.*, “ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification,” *Proceedings of the Web Conference (WWW)*, 2022, representative Transformer-based traffic classifier.
- [8] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] M. Ring, S. Wunderlich, D. Gröndler *et al.*, “A survey of network-based intrusion detection data sets,” in *Computers & Security*, vol. 86, 2019, pp. 147–167.
- [10] IETF, “RFC 9001: Using TLS to secure QUIC,” <https://www.rfc-editor.org/rfc/rfc9001>, 2021.
- [11] K. P. Dyer, S. Isakov, T. Lee *et al.*, “Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail,” in *IEEE Symposium on Security and Privacy (Oakland)*, 2012.
- [12] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, “Effective attacks and provable defenses for website fingerprinting,” in *USENIX Security Symposium*, 2014.
- [13] J. Hayes and G. Danezis, “k-fingerprinting: A robust scalable website fingerprinting technique,” in *USENIX Security Symposium*, 2016.
- [14] A. Shusterman, A. Cohen, N. Nissim *et al.*, “Cachebrowser: A DNS-based website fingerprinting attack,” *IEEE/ACM Transactions on Networking*, vol. 29, no. 4, pp. 1623–1636, 2021.
- [15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.
- [16] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [17] IETF, “RFC 9000: QUIC: A UDP-based multiplexed and secure transport,” <https://www.rfc-editor.org/rfc/rfc9000>, 2021.

APPENDIX A REPRODUCIBILITY

This appendix lists artifacts, evaluation commands, and environment details for reproducing the numbers cited in Section V. A machine-readable checklist is maintained in `phase5/REPRODUCIBILITY.md`.

A. Dataset and splits

We use CESNET-QUIC22-XS [1]: QUIC flows collected on a national research-and-education backbone. Training uses calendar week W-2022-44; testing uses the subsequent week W-2022-45 (49,305 flows, 64 application classes after filtering classes with fewer than 200 training samples). Each flow is a (3, 30) tensor of IPT, direction, and normalized packet size.

B. Checkpoints and artifacts

Table V summarizes primary files. Obfuscated tensors are stored as `phase3/artifacts/obfuscated_<policy>.pt`; overhead is logged in `phase3/artifacts/obfuscation_manifest.json`.

TABLE V: Key reproducibility artifacts.

Item	Relative path
Test tensors	phase1/artifacts/test_tensors.pt
Transf. ckpt	phase2/artifacts/transformer_production.pt
BiLSTM ckpt	phase2/artifacts/cnn_bilstm_best.pt
Obfuscated	phase3/artifacts/obfuscated*.pt
Results CSV	phase4/results/accuracy_results.csv
Bootstrap CIs	phase4/bootstrap_ci.csv

C. Evaluation commands

Tier A/B (Transformer sweep):

```
cd phase4
python run_experiments.py
--device cuda
--checkpoint
./phase2/artifacts/
transformer_production.pt
```

Tier C (bootstrap / ablation / arch.):

```
python run_tier_c.py --all
--device cuda
--checkpoint
./phase2/artifacts/
transformer_production.pt
python run_tier_c.py --bilstm
--compare-architectures
--device cuda
--bilstm-checkpoint
./phase2/artifacts/
cnn_bilstm_best.pt
```

Re-running `run_tier_c.py --all` regenerates bootstrap CIs, paired tests, and channel-ablation tables referenced in Section V.

D. Environment and ethics

Experiments were run with pinned dependencies in `phase4/requirements.txt`. CESNET-QUIC22 is distributed under the Zenodo terms cited in [1]; we use only flow metadata (sizes, directions, timing) and application labels—no payload content is accessed or stored in our pipeline.